

1. GIT & GITHUB FUNDAMENTALS

★ What is Git ?

Git is a distributed version control system. It helps developers track changes in code, collaborate with others and manage different versions of a project efficiently.

★ Why Developers Use Git ?

- Track every change in the project.
- Work on new features without disturbing main code.
- Collaborate with team members seamlessly.
- Revert to previous versions if something breaks.
- Keep a backup of code in remote repositories.

★ What is GitHub ?

GitHub is a cloud based platform that hosts Git repositories. It provides tools for collaboration, code review, issue tracking and project management.

Interview Tip

Git is a tool that tracks changes in your code locally.

GitHub is a platform that stores your Git repositories online.

★ Version Control vs Source Control

Version Control (Git)	Source Control
• Tracks every change (history). • We can revert to any previous version. • Allows offline work. • Example : <u>Git</u>	• Only latest version is available. • Cannot go back to previous versions. • Requires constant connection. • Example : <u>CVS</u>, <u>Subversion</u>

★ Centralized vs Distributed Version Control

- Centralized (e.g. SVN) : Code is stored on a central server. If server is down, developers cannot work.
- Distributed (e.g. Git) : Every developer has a local copy of the entire repository. Work can continue offline.

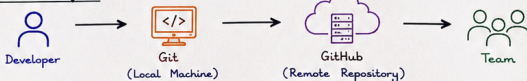
★ Why Git is Important in Teams ?

- Multiple developers can work on the same project without overwriting each other's work.
- Easy collaboration using branches and pull requests.
- Maintains a complete project history.
- Helps in code review and quality improvement.

★ Benefits of Git

- Distributed - everyone has a local copy.
- Fast and lightweight.
- Safe - no single point of failure.
- Flexible branching and merging.
- Widely used in industry.

★ Typical Workflow



Key Takeaway



Git helps you track and manage changes in your code locally.

GitHub helps you store your code in the cloud and collaborate with others.

2. HOW GIT WORKS - ARCHITECTURE

★ Components of Git

Git has 4 main areas where your code and changes move from one place to another.

① Working Directory

This is the local folder on your computer where you create, edit, delete files. All the changes happen here.

② Staging Area (Index)

This is a temporary area where you add the changes using 'git add'. It tells Git which changes you want to include in the next commit.

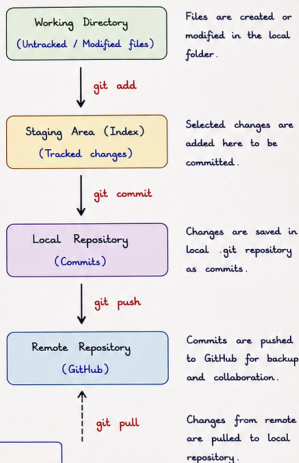
③ Local Repository

This is the .git folder on your machine. When you commit, Git stores the changes here with a unique ID (commit hash).

④ Remote Repository (GitHub)

This is the copy of your project that is hosted on GitHub (cloud). You push your commits from local repo to remote repo.

★ Git Workflow



★ Basic Commands (First Steps)

Command	Syntax	Description
git init	git init	Initializes a new Git repository in current folder.
git status	git status	Shows the current status of files.
git add	git add <file>	Adds a specific file to staging area.
	git add .	Adds all changes to staging area.
git commit	git commit -m "message"	Commits the staged changes to local repo.

★ Key Points to Remember

- Changes exist first in Working Directory.
- We use 'git add' to move changes to Staging Area.
- We use 'git commit' to save changes to Local Repo.
- We use 'git push' to upload changes to GitHub.
- We use 'git pull' to download changes from GitHub.

INTERVIEW TIP



Git is a distributed version control system. It tracks changes locally and allows multiple developers to work on the same project without affecting each other.

Important Notes

- A commit is a snapshot of your project at a specific time.
- Every commit has a unique SHA (hash).
- You can always go back to any previous version using commits.
- Git does not store differences, it stores snapshots.
- Git works offline. You can commit without internet.

3. ESSENTIAL GIT COMMANDS

Git commands help us to track changes, manage versions and collaborate efficiently.

Command	Syntax	Description	Example
<code>git clone</code>	<code>git clone <url></code>	Clones a remote repository into your local machine.	<code>git clone https://github.com/user/repo.git</code>
<code>git init</code>	<code>git init</code>	Initializes a new Git repository in current folder.	<code>git init</code>
<code>git status</code>	<code>git status</code>	Shows the status of files in working directory.	<code>git status</code>
<code>git add</code>	<code>git add <file></code> <code>git add .</code>	Adds a file or all files to the staging area.	<code>git add index.html</code> <code>git add .</code>
<code>git commit</code>	<code>git commit -m "message"</code>	Commits the staged changes to local repository.	<code>git commit -m "Initial commit"</code>
<code>git push</code>	<code>git push origin <branch></code>	Pushes committed changes to remote repository.	<code>git push origin main</code>
<code>git pull</code>	<code>git pull origin <branch></code>	Fetches and merges changes from remote repository.	<code>git pull origin main</code>
<code>git fetch</code>	<code>git fetch origin</code>	Fetches changes from remote but doesn't merge.	<code>git fetch origin</code>
<code>git log</code>	<code>git log</code>	Shows commit history.	<code>git log</code>
<code>git diff</code>	<code>git diff</code>	Shows unstaged changes.	<code>git diff</code>
<code>git diff --staged</code>	<code>git diff --staged</code>	Shows changes between staging area and last commit.	<code>git diff --staged</code>
<code>git restore</code>	<code>git restore <file></code> <code>git restore --staged <file></code>	Discards changes in working dir or unstage a file.	<code>git restore index.html</code> <code>git restore --staged index.html</code>
<code>git rm</code>	<code>git rm <file></code>	Removes file from working dir and staging area.	<code>git rm old.txt</code>
<code>git mv</code>	<code>git mv old.txt new.txt</code>	Renames or moves a file.	<code>git mv old.txt new.txt</code>
<code>git show</code>	<code>git show <commit_id></code>	Shows details of a commit.	<code>git show a1b2c3d</code>

Helpful Shortcuts

- `git config --global user.name "Your Name"` → Set username
- `git config --global user.email "you@email.com"` → Set email
- `git config --list` → Show configuration
- `git branch` → List all branches
- `git remote -v` → Show remote repositories
- `git log --online --graph` → Visualize history
- `git log --stat` → Summary of changes

INTERVIEW TIP

- `git add` moves changes to staging area.
- `git commit` saves changes in local repo.
- `git push` uploads changes to GitHub.
- `git pull` = `fetch` + `merge`.
- `fetch` gets changes, `pull` applies them.

☆ Pro Tip

Always run `git status` before committing. It helps you understand what will be committed.

4. BRANCHING & MERGING

★ What is a Branch ?

A **branch** in Git is like an independent line of development. It allows you to work on new features, fix bugs or experiment without affecting the main codebase.

★ Why use Branches ?

- Work on new features without breaking main code.
- Easy collaboration in teams.
- Test and experiment safely.
- Keep production code stable.
- Merge changes only when ready.

★ Important Branch Commands

<code>git branch</code>	List all branches
<code>git branch <name></code>	Create a new branch
<code>git checkout <name></code>	Switch to a branch
<code>git switch <name></code>	Switch to a branch (new syntax)
<code>git branch -d <name></code>	Delete a branch
<code>git branch -D <name></code>	Force delete a branch
<code>git checkout -b <name></code>	Create and switch to new branch

★ Merge Conflicts

A merge conflict happens when Git cannot automatically decide which changes to keep.

- Git will mark conflicting files.
- You need to open the file, resolve conflicts manually.
- Then add and commit the changes.
- After resolving:

`git add <file>` → `git commit`

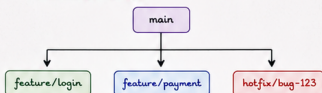
★ Best Practices

- ✓ Create a new branch for every feature or bug fix.
- ✓ Keep your branches small and focused.
- ✓ Pull latest changes from main before starting work.
- ✓ Write meaningful commit messages.

Types of Branches

- **main (or master)** : Production ready code.
- **feature/*** : New features.
- **release/*** : Preparing for a release.
- **hotfix/*** : Urgent bug fixes in production.
- **develop** : Integration branch for features.

Branching Example



All branches are created from main and later merged back when work is complete.

★ Merging Branches

Merging integrates changes from one branch into another.
Command :

```
git checkout main # switch to target branch
git merge feature/login # merge source branch
```

After merging, changes from feature/login will be part of main branch.

★ Check Branch Status

<code>git status</code>	Show current branch and status
<code>git branch</code>	List all local branches
<code>git branch -r</code>	List all remote branches
<code>git branch -a</code>	List all branches (local + remote)
<code>git log --oneline --graph --all</code>	Visualize all branches

INTERVIEW TIP

Branches help in parallel development. They keep the main code stable while allowing multiple features to be built simultaneously.

Golden Rule:

Commit to your branch,
Merge to main.



5. MERGE vs REBASE

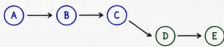
Both **MERGE** and **REBASE** are used to combine changes from one branch into another, but they do it in different ways.

1) MERGE

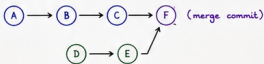
- Combines branches by creating a new merge commit.
- Preserves the complete history.
- Safe to use on shared branches.

Example: ☐ main ☐ feature

Before Merge



After Merge



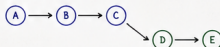
Command: `git checkout main`
`git merge feature` # merges feature into main

2) REBASE

- Moves or "replays" your branch commits on top of another branch.
- Creates a linear history (no merge commit).
- Rewrites history - use with caution.

Example: ☐ main ☐ feature

Before Rebase



After Rebase



Command: `git checkout feature`
`git rebase main` # reapplies feature commits on top of main

★ Key Differences

Aspect	Merge	Rebase
History	Preserves history (with merge commit)	Creates linear history
Commit IDs	No change in existing commits	Changes commit IDs
Safety	Safe on shared branches	Not safe on shared branches
Use Case	When you want full history	When you want a clean history
Command	<code>git merge</code>	<code>git rebase</code>

3) CHERRY-PICK

Apply a specific commit from one branch to another.

Example:



Command: `git checkout main`
`git cherry-pick <commit-hash-of-Q>` # applies commit Q on main

When to use what?

- Use **MERGE** when working on team branches or shared branches.
- Use **REBASE** when working on your own feature branch before creating Pull Request.
- Never rebase** commits that are already pushed and shared.



Important Caution

- Rebase changes history by creating new commits.
- If you rebase a branch that others have pulled, you will cause problems for them.
- Rule of Thumb**: **REBASE** your private work, **MERGE** your shared work.



INTERVIEW TIP

- Merge keeps history and is safer.
- Rebase keeps history clean but rewrites commits.
- Always prefer merge on main or shared branches.

Remember:

"Never rebase what you have pushed."

6. GITHUB WORKFLOW & COLLABORATION

★ What is GitHub ?

GitHub is a **cloud platform** that hosts Git repositories.

It provides tools for collaboration, code review, issue tracking, project management and CI/CD.

Key Terms

Repository (Repo) : A project folder that contains all files and revision history.

Fork : A copy of someone else's repository.

Clone : A copy of the repository on your local machine.

Pull Request (PR) : Request to merge changes into another branch.

Issue : Used to track tasks, bugs or feature requests.

★ Typical GitHub Collaboration Workflow



★ Step-by-Step Explanation

1. **Fork** : Create your own copy of the repository on GitHub.
2. **Clone** : Clone the forked repository to your local machine.
3. **Create Branch** : Create a new branch for your feature or fix.
4. **Make Changes** : Edit files, add new code or fix bugs.
5. **Commit** : Save your changes locally with a meaningful commit message.
6. **Push** : Push your branch to your forked repository on GitHub.
7. **Pull Request** : Open a pull request to the original repository.
8. **Code Review** : Team reviews your code and suggests changes.
9. **Merge** : Once approved, your changes are merged into the main branch.
10. **Delete Branch** : Delete the branch after successful merge.

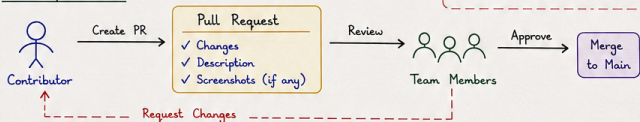
Why Pull Request (PR) ?

- Discuss and review the changes.
- Maintain code quality.
- Avoid conflicts.
- Track all the changes.
- Safe way to contribute to a project.

Best Practices

- ✓ Create small and focused PRs.
- ✓ Write clear commit messages.
- ✓ Keep your branch updated.
- ✓ Review before merging.
- ✓ Delete merged branches.

★ Pull Request Flow



★ Useful GitHub Features

- **Issues** : Track tasks, bugs and feature requests.
- **Projects** : Organize work using boards and columns.
- **Labels** : Categorize issues and pull requests.
- **Milestones** : Group issues into a release.
- **Actions** : Automate workflows using CI/CD.
- **Wiki** : Add documentation for your project.
- **Discussions** : Discuss ideas and ask questions.



INTERVIEW TIP

GitHub is not just for hosting code.

It helps teams collaborate efficiently, maintain quality and ship better software faster.

7. UNDO OPERATIONS & GIT BEST PRACTICES

★ Undo Operations in Git

Git provides multiple ways to undo changes. Each command is useful in different situations.

Command	Use	Scope	Safe on Shared Repo?
git reset	Undo commits by moving HEAD	Commits	No (if used improperly)
git revert	Undo commits by creating a new commit	Commits	Yes (Safe)
git restore	Undo changes in working directory or staging area	Files	Yes
git stash	Temporarily save changes without committing	Changes	Yes

1) git reset

Moves the HEAD to a previous commit.

- Soft Reset (keeps changes in staging area)

```
git reset --soft <commit-hash>
```

- Mixed Reset (keeps changes in working dir)

```
git reset --mixed <commit-hash> (default)
```

- Hard Reset (discard all changes)

```
git reset --hard <commit-hash>
```

2) git revert

Creates a new commit that undoes the changes of a previous commit.

```
git revert <commit-hash>
```

3) git restore

Restore changes in working directory or staging area.

- Discard changes in working directory

```
git restore <file>
```

- Unstage a file (move from staging to working dir)

```
git restore --staged <file>
```

4) git stash

Save your changes temporarily and clean working directory.

- Stash changes

```
git stash
```

- Apply stash

```
git stash apply
```

- Apply and remove stash

```
git stash pop
```

- List all stashes

```
git stash list
```

- Drop stash

```
git stash drop
```

Stash Workflow

Work on something but not ready to commit

↓
git stash

↓
Do other work

↓
git stash apply / pop

↓
Continue your work

★ Best Practices

- ✓ Commit frequently with meaningful messages.
- ✓ Pull latest changes before starting work.
- ✓ Keep your branches small and focused.
- ✓ Use feature branches for new work.
- ✓ Never push directly to main.
- ✓ Review code via Pull Requests.
- ✓ Use .gitignore to avoid committing unnecessary files.
- ✓ Do not commit passwords, API keys or secrets.
- ✓ Keep your repository clean.

Hard Reset vs Soft Reset

Feature	Soft Reset	Hard Reset
Moves HEAD	Yes	Yes
Keeps changes in staging area	Yes	No
Keeps changes in working directory	Yes	No
Deletes commits	No	Yes
Safe on shared repo	No	No



INTERVIEW TIP

Revert is safer than reset for shared repositories.
Reset rewrites history, revert preserves history.

Golden Rule

Always think before you reset.
Once pushed, history changes affect others.

8. GIT INTERVIEW CHEAT SHEET

★ Key Comparisons

Concept	Difference
Git vs GitHub	Git is a distributed version control system. GitHub is a cloud platform to host Git repositories.
Fetch vs Pull	Fetch downloads changes from remote. Pull = Fetch + Merge (downloads and merges).
Merge vs Rebase	Merge preserves history by creating a merge commit. Rebase creates a linear history by rewriting commits.
Reset vs Revert	Reset moves HEAD to a previous commit and can discard changes. Revert creates a new commit that undoes changes.
Fork vs Clone	Fork creates a copy on GitHub server. Clone creates a copy on your local machine.
Commit vs Push	Commit saves changes in local repository. Push uploads changes to remote repository.
Branch vs Tag	Branch is movable pointer to a series of commits. Tag is fixed reference to a specific commit.
Local vs Remote Repo	Local repo exists on your machine. Remote repo exists on a server (e.g., GitHub).

★ Top 20 Rapid-Fire Questions

1. What is Git?
2. Why is Git a distributed system?
3. What is the difference between Git and GitHub?
4. What is the purpose of .gitignore file?
5. What is a repository?
6. What is the staging area?
7. What is a commit?
8. What does HEAD refer to?
9. What is origin?
10. What is the difference between fetch and pull?
11. What is the difference between merge and rebase?
12. What is cherry-pick?
13. What is stash and why is it used?
14. What is detached HEAD state?
15. What is a fast-forward merge?
16. How do you resolve merge conflicts?
17. What is a pull request?
18. What is a squash commit?
19. What is tagging in Git?
20. Explain a typical Git workflow.

★ Typical Git Workflow



★ Useful Git Tips

- Use `git status` frequently to know what's happening.
- Pull latest changes before you start working.
- Write meaningful `commit messages`.
- Keep your commits `small and atomic`.
- Review your code via `Pull Requests`.
- **Never** commit sensitive information (passwords, API keys).
- Use `.gitignore` to keep unnecessary files out of repo.
- Keep your repository clean and organized.
- Delete branches after merging.
- Communication + good workflow = great teamwork!

Common .gitignore Entries

<code>.class</code>	<code>*.jar</code>
<code>.log</code>	<code>*.war</code>
<code>.env</code>	<code>.idea/</code>
<code>node_modules/</code>	<code>.vscode/</code>
<code>target/</code>	<code>build/</code>

Commit Message Format

```
<type>(<scope>): <subject>
[optional body]
[optional footer]
```

Example:

```
feat(auth): add login API
→ feat : new feature
→ fix : bug fix
→ docs : documentation
→ chore : maintenance
```



Interview Tip

Employers don't just test commands.
They test your understanding of concepts and real-world usage.
Practice. Use Git daily. Build intuition!

Remember

Good Git habits today
save you from big
problems tomorrow.

